

# MODULE ALGORITHMIQUE ET STRUCTURES DE DONNÉES AVANCÉES

## *ALGORITHMES DE TRI*

©B. DERDOURI

CH2-Algorithmes de tris L3I-S5-U-CERGY

9/10/2023

1

### PLAN DE COURS

- **Définition :Tri d'un tableau**
- **Importance du tri**
- **Analyse des algorithmes**
- **Tri par insertion**
- **Tri par sélection**
- **Tri fusion**
- **Tri rapide**

©B. DERDOURI

CH2-Algorithmes de tris L3I-S5-U-CERGY

9/10/2023

2

## ALGORITHMES DE TRI

- **Définition : Tri d'un tableau**

- Soit  $\text{tab}[1..n]$  un tableau composé de  $n$  éléments à valeurs dans un ensemble  $E$  muni d'une relation d'ordre notée  $\leq$ .
- Trier le tableau  $\text{tab}$  consiste à construire un tableau  $\text{tab}'$  tel que :
  - $\text{Tab}'$  soit trié
  - $\text{Tab}$  et  $\text{tab}'$  contiennent les mêmes éléments.

- **Remarque**

- Le plus souvent  $\text{tab}$  et  $\text{tab}'$  sont le même tableau : on construit  $\text{tab}'$  en permutant entre eux les éléments de  $\text{tab}$ .

## ALGORITHMES DE TRI

- **Importance du tri**

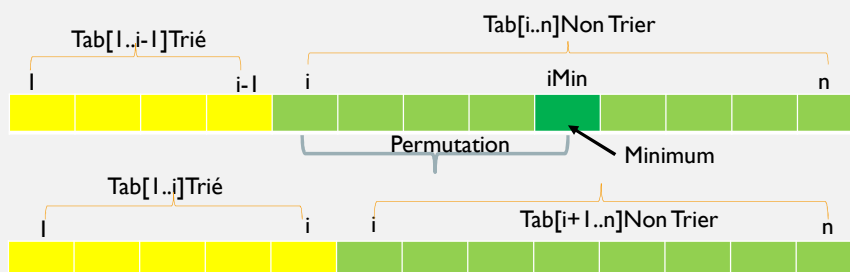
- Il est intéressant de trier pour faciliter des recherches, par exemple par ordre alphabétique.
- Dans beaucoup d'algorithmes il faut maintenir de manière dynamique des tableaux triés, pour trouver rapidement les plus petits et plus grands éléments.

## ALGORITHMES DE TRI

- **Analyse de l'algorithme**
- **Données**
  - $n$  : Entier (=nombre d'éléments présents dans tab)
  - tab : Tableau de TElement à trié
- **Traitement**
  - application de L'algorithme de tri
- **Résultat**
  - tab : tableau trié

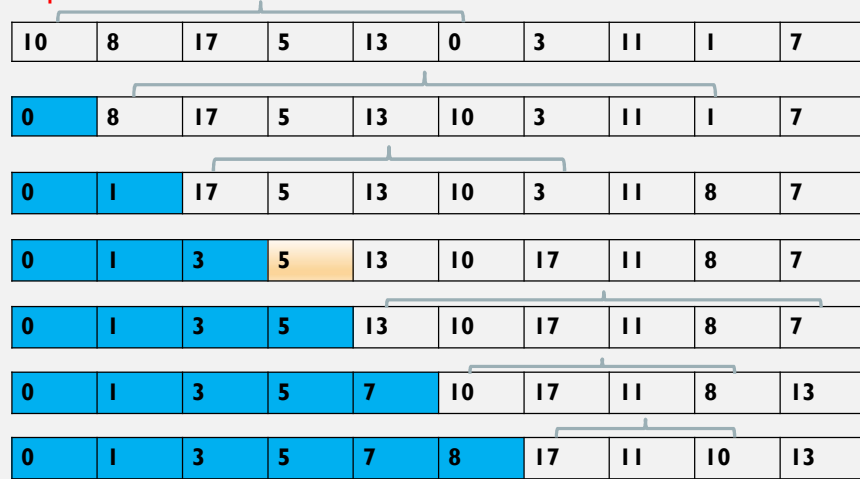
## TRI PAR SÉLECTION

- **Principe de l'algorithme**
- Trier un tableau composé de  $n$  éléments c'est extraire le minimum, le permuter avec l'élément courant puis trier les  $(n-1)$  éléments restants du tableau.



## TRI PAR SÉLECTION

- Exemple



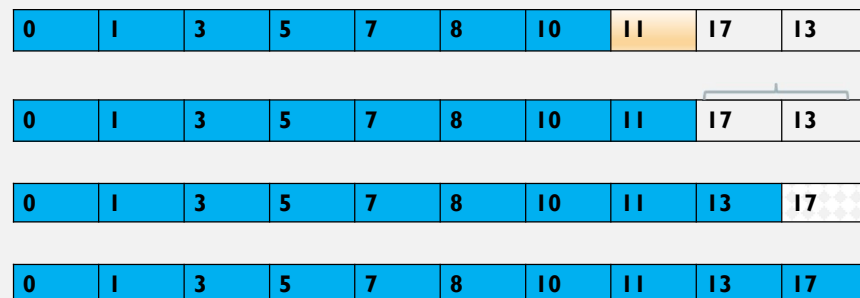
©B. DERDOURI

CH2-Algorithmes de tris L3I-S5-U-CERGY

9/10/2023

7

- Exemple suite :



©B. DERDOURI

CH2-Algorithmes de tris L3I-S5-U-CERGY

9/10/2023

8

## TRI PAR SÉLECTION

- Version itérative de l'algorithme

```
procédure itTriSelection(n, tab)
P.F      n : Entier (E)
         tab : tableau (E/S)
Début
  Pour i variant de 1 à (n-1) faire
    iMin ← indMin(i, n, tab)
    Si i ≠ iMin alors
      permutation(i, iMin, tab)
    Fsi
  Fpour
Fin
```

```
// Permutation tab[i] avec tab[j]
Procédure permutation(i, j, tab)
P.F      i, j : Entier (E)
         tab : tableau (E/S)
Début
  tmp ← tab[i]
  tab[i] ← tab[j]
  tab[j] ← tmp
Fin
```

## VERSION RÉCURSIVE DU TRI PAR SÉLECTION

- Rappel : Définition Réursive d'un tableau**
- Un tableau composé de n éléments est
  - Soit vide
  - Soit composé d'un élément et d'un sous tableau à (n-1) éléments.
- On distingue deux versions de l'algorithme de tri selon le sens de parcours du tableau.

## VERSION RÉCURSIVE DU TRI PAR SÉLECTION

- **Version 1 : parcours du tableau selon le sens décroissant des indices (Droite → Gauche).**
- **Définition Réursive d'un tableau**
- Un tableau  $\text{tab}[1..n]$  composé de  $n$  éléments est
  - Soit vide : c'est-à-dire  $n=0$
  - Soit composé d'un élément le dernier  $\text{tab}[n]$  et d'un sous tableau à  $(n-1)$  éléments  $\text{tab}[1..n-1]$ .
- **Remarque**
  - Un tableau vide ou composé d'un élément est trié

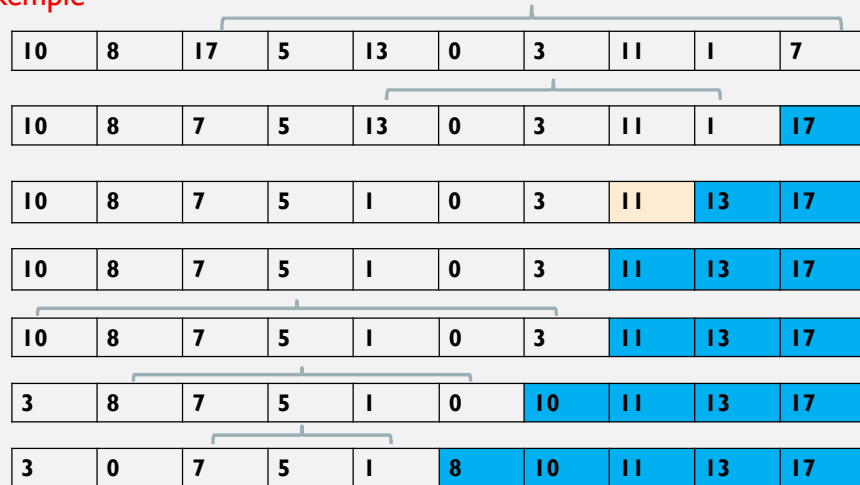
## VERSION RÉCURSIVE DU TRI PAR SÉLECTION

- **Raisonnement**
- **Si** le tableau est vide ( $n=0$ ) ou composé d'un élément ( $n=1$ ), il est considéré comme trié.
 

→ C'est donc le **cas d'arrêt** de la récursivité.
- **Sinon : cas général**
  - Extraire l'indice du maximum ( $iMax$ ) dans le sous tableau  $\text{tab}[1..n]$  non trié.
  - On effectue la permutation si le maximum ( $\text{tab}[iMax]$ ) est différent du dernier élément ( $\text{tab}[n]$ ) du sous tableau.
  - On effectue un appel récursif sur les  $(n-1)$  éléments restants. C'est-à-dire on effectue l'appel récursif sur  $\text{tab}[1..n-1]$ .

## VERSION RÉCURSIVE DU TRI PAR SÉLECTION

- Exemple



©B. DERDOURI

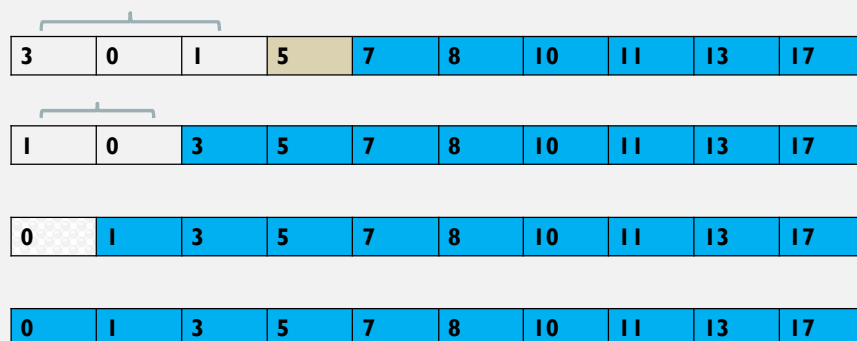
CH2-Algorithmes de tris L3I-S5-U-CERGY

9/10/2023

13

## VERSION RÉCURSIVE DU TRI PAR SÉLECTION

- Exemple suite :



©B. DERDOURI

CH2-Algorithmes de tris L3I-S5-U-CERGY

9/10/2023

14

## VERSION RÉCURSIVE DU TRI PAR SÉLECTION

- **Version 1 : parcours du tableau selon le sens décroissant des indices (Droite → Gauche).**

```

Procédure reTriSelection_VI(n, tab)
P.F      n : Entier (E)
         tab : Tableau (E /S)
Début    // On effectue le tri si le tableau est composé au moins de 2 élts
         Si n > 1 alors
             // Extraction de l'indice du Maximum dans tab[1..n]
             iMax ← indiceMax(1, n, tab)
             si n ≠ iMax alors // On effectue la Permutation
                 permutation(iMax, n, tab)
             fsi
             //appel récursif
             reTriSelection_VI(n-1, tab)
         fin
Fin

L'appel de la procédure est : reTriSelection_VI(n, tab)

```

## VERSION RÉCURSIVE DU TRI PAR SÉLECTION

- **Version 2 : parcours du tableau selon le sens croissant des indices.**
- **Définition Récursive d'un tableau**
- Un tableau  $\text{tab}[i..n]$  composé de  $(n-i+1)$  éléments est
  - Soit vide : c'est-à-dire  $i > n$
  - Soit composé d'un élément le premier  $\text{tab}[i]$  et d'un sous tableau à  $(n-i)$  éléments  $\text{tab}[i+1..n]$ .
- **Remarque**
- Un tableau vide ( $i > n$ ) ou composé d'un élément ( $i = n$ ) est trié



## VERSION RÉCURSIVE DU TRI PAR SÉLECTION

- **Raisonnement**
- **Si** le tableau est vide ( $i > n$ ) ou composé d'un élément ( $i = n$ ) est considéré comme trié. → C'est donc le **cas d'arrêt** de la récursivité.
- **Sinon : cas général**
  - Extraire le minimum dans le tableau  $\text{tab}[i..n]$
  - On effectue la permutation si le minimum est différent du premier élément  $\text{tab}[i]$  du sous tableau.
  - On effectue un appel récursif sur les éléments restants. C'est-à-dire on effectue l'appel récursif sur  $\text{tab}[i+1..n]$ .

## VERSION RÉCURSIVE DU TRI PAR SÉLECTION

- **Version 2 : parcours du tableau selon le sens croissant des indices (Gauche → Droite).**

```

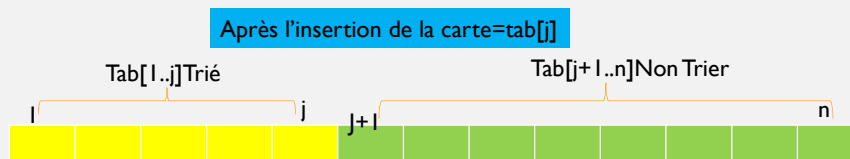
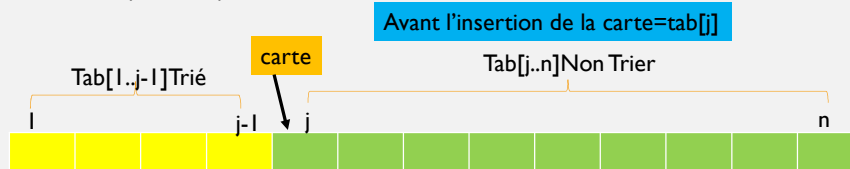
Procédure reTriSelection_V2(i, n, tab)
P.F      i, n : Entier (E)
         tab : Tableau (E / S)
Début    // On effectue le tri si le tableau est composé au moins de 2 élt (i < n)
         Si i < n alors // Extraction de l'indice du Minimum ds tab[i..n]
             iMin ← IndiceMin(i, n, tab)
             si i ≠ iMin alors // On effectue la Permutation
                 permutation(i, iMin, tab)
             fsi
             //appel récursif
             reTriSelection_V2(i+1, n, tab)
         finsi
Fin

L'appel de la procédure est : reTriSelection_V2(1, n, tab)

```

## TRI PAR INSERTION

- **Principe de l'algorithme**
- Cet algorithme de tri est basé directement sur l'insertion d'un élément (=carte) dans un tableau trié.



## TRI PAR INSERTION

### **Mécanisme d'insertion**

- On commence la recherche de la place de la carte= $\text{tab}[j]$  par la fin du sous tableau trié pour pouvoir effectuer en même temps le décalage de la fin du tableau, ce qui est nécessaire avant de faire l'insertion.
- On recherche la place où insérer le nouvel élément de **manière séquentielle**, on dit alors que l'on fait de l'insertion séquentielle. Dans ce cas, on dit que le tri par insertion utilise une approche séquentielle (incrémentale).

## TRI PAR INSERTION

- Exemple d'insertion d'un élément dans un tableau trié **0=carte à insérer**

2 4 7 10 13 18 **Place libre d'insertion**

- $0 < 18$  → on décale le 18
- $0 < 13$  → on décale le 13
- $0 < 10$  → on décale le 10
- $0 < 7$  → on décale le 7
- $0 < 4$  → on décale le 4
- $0 < 2$  → on décale le 2
- On réalise l'insertion**

|   |   |   |    |    |    |    |    |
|---|---|---|----|----|----|----|----|
| 2 | 4 | 7 | 10 | 13 | 18 |    | 18 |
| 2 | 4 | 7 | 10 | 13 |    | 13 | 18 |
| 2 | 4 | 7 | 10 |    | 10 | 13 | 18 |
| 2 | 4 | 7 |    | 7  | 10 | 13 | 18 |
| 2 | 4 |   | 4  | 7  | 10 | 13 | 18 |
| 2 |   | 2 | 4  | 7  | 10 | 13 | 18 |
| 0 | 2 | 4 | 7  | 10 | 13 | 18 |    |

©B. DERDOURI

CH2-Algorithmes de tris L3I-S5-U-CERGY

9/10/2023

21

## TRI PAR INSERTION

- Exemple d'insertion d'un élément dans un tableau trié **9=carte à insérer**

2 4 7 10 13 18 **Place libre d'insertion**

- $9 < 18$  → on décale le 18
- $9 < 13$  → on décale le 13
- $9 < 10$  → on décale le 10
- $9 \geq 7$  → **On réalise l'insertion**

|   |   |   |    |    |    |    |    |
|---|---|---|----|----|----|----|----|
| 2 | 4 | 7 | 10 | 13 | 18 |    | 18 |
| 2 | 4 | 7 | 10 | 13 |    | 13 | 18 |
| 2 | 4 | 7 | 10 |    | 10 | 13 | 18 |
| 2 | 4 | 7 | 9  | 10 | 13 | 18 |    |

©B. DERDOURI

CH2-Algorithmes de tris L3I-S5-U-CERGY

9/10/2023

22

## TRI PAR INSERTION

- Exemple d'insertion d'un élément dans un tableau trié

|   |   |   |    |    |    |  |
|---|---|---|----|----|----|--|
| 2 | 4 | 7 | 10 | 13 | 18 |  |
|---|---|---|----|----|----|--|

29=carte  
à insérer

Place libre d'insertion

- 29 >= 18 →

On réalise  
l'insertion

|   |   |   |    |    |    |    |
|---|---|---|----|----|----|----|
| 2 | 4 | 7 | 10 | 13 | 18 | 29 |
|---|---|---|----|----|----|----|

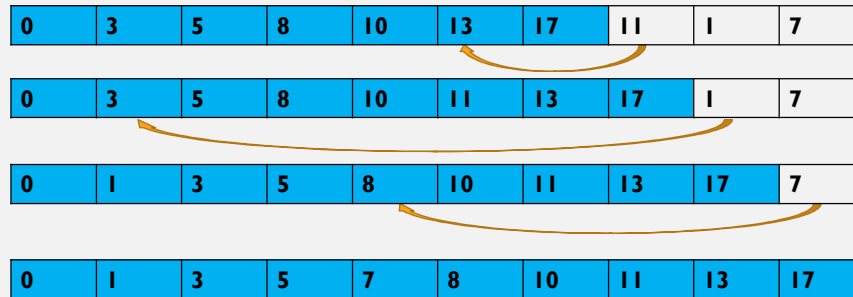
## TRI PAR INSERTION

- Exemple

|    |    |    |    |    |    |   |   |    |   |   |
|----|----|----|----|----|----|---|---|----|---|---|
| 10 | 8  |    | 17 | 5  | 13 | 0 | 3 | 11 | 1 | 7 |
| 8  | 10 |    | 17 | 5  | 13 | 0 | 3 | 11 | 1 | 7 |
| 8  | 10 | 17 |    | 5  | 13 | 0 | 3 | 11 | 1 | 7 |
| 5  | 8  | 10 | 17 |    | 13 | 0 | 3 | 11 | 1 | 7 |
| 5  | 8  | 10 | 13 | 17 |    | 0 | 3 | 11 | 1 | 7 |
| 0  | 5  | 8  | 10 | 13 | 17 |   | 3 | 11 | 1 | 7 |

## TRI PAR INSERTION

- Exemple suite :



## VERSION ITÉRATIVE DU TRI PAR INSERTION

```

Procédure triInsertion(n, tab)
P.F      n : Entier (E)
        tab : Tableau (E / S)
Début  // On commence le tri à partir de la 2ème carte
      pour j variant de 2 à n faire
        // insertion de la carte dans une main triée tab[1..j-1]
        carte ← tab[j]
        i ← j-1
        // On cherche la place d'insertion
        tantque i>0 et carte < tab[i] faire
          tab[i+1] ← tab[i]
          i ← i-1
        finTantque
        //on effectue l'insertion
        tab[i+1] ← carte
      finPour
Fin
    
```

L'appel de la procédure est : triInsertion(n, tab)